

```
--- djangoapi\scripts\crop\DjangoModels\tablasDJ.py ---
```

```
from datetime import date, datetime
```

```
from django.db import connection
```

```
from django.contrib.gis.geos import GEOSGeometry
```

```
from django.forms.models import model_to_dict
```

```
from djangoapi.settings import EPSG_FOR_GEOMETRIES, ST_SNAP_PRECISION
```

```
class TablasDJ:
```

```
    def __init__(self, model):
```

```
        self.model = model
```

```
        self.srid = EPSG_FOR_GEOMETRIES
```

```
        self.precision = ST_SNAP_PRECISION
```

```
    def _get_snapped_wkb(self, wkt):
```

```
        """Aplica ST_SnapToGrid vía SQL para limpiar la geometría."""
```

```
        with connection.cursor() as cur:
```

```
            query = "SELECT ST_SnapToGrid(ST_GeomFromText(%s, %s), %s)"
```

```
            cur.execute(query, [wkt, self.srid, self.precision])
```

```
            return cur.fetchone()[0]
```

```
    def _check_interior_intersection(self, wkb_geom, exclude_id=None):
```

```
        """
```

```
        Verifica intersección de interiores excluyendo el ID actual si se proporciona.
```

```
        """
```

```
        table_name = self.model._meta.db_table
```

```
        query = f"SELECT id FROM {table_name} WHERE ST_Relate(geom, %s, 'T*****')"
```

```
        params = [wkb_geom]
```

```
        # Si estamos en un UPDATE, añadimos la exclusión del propio ID
```

```
        if exclude_id:
```

```
            query += " AND id != %s"
```

```
            params.append(exclude_id)
```

```
        with connection.cursor() as cur:
```

```
            cur.execute(query, params)
```

```
            return cur.fetchall()
```

```
    def selectAll(self, id_min=0):
```

```
        """Retorna la cantidad de registros con id mayor al indicado."""
```

```
        return self.model.objects.filter(id__gt=id_min).count()
```

```
    def selectAsDict(self, id_min=0):
```

```
        """Retorna los registros como una lista de diccionarios con WKT."""
```

```
        qs = self.model.objects.filter(id__gt=id_min)
```

```
        resultados = []
```

```
        for obj in qs:
```

```
            d = model_to_dict(obj)
```

```
            d['geom'] = obj.geom.wkt if obj.geom else None
```

```
            for field in obj._meta.fields:
```

```
                value = getattr(obj, field.name)
```

```
                if isinstance(value, datetime):
```

```

        d[field.name] = value.strftime("%Y-%m-%d %H:%M:%S")
    elif isinstance(value, date):
        d[field.name] = value.strftime("%Y-%m-%d")
    resultados.append(d)
return resultados

def selectAsTuple(self, id_min=0):
    """Retorna los registros como una lista de tuplas con geometria en WKT."""
    qs = self.model.objects.filter(id__gt=id_min)
    resultados = []

    for obj in qs:
        fila = []
        for field in obj._meta.fields:
            value = getattr(obj, field.name)
            if isinstance(value, GEOSGeometry):
                fila.append(value.wkt if value else None)
            elif isinstance(value, datetime):
                fila.append(value.strftime("%Y-%m-%d %H:%M:%S"))
            elif isinstance(value, date):
                fila.append(value.strftime("%Y-%m-%d"))
            else:
                fila.append(value)
        resultados.append(tuple(fila))

    return resultados

def delete(self, d: dict):
    """Elimina un registro por ID tras verificar su existencia."""
    try:
        obj = self.model.objects.get(id=d['id'])
        obj.delete()
        return {'ok': True, 'message': f'Registro {d["id"]} eliminado'}
    except self.model.DoesNotExist:
        return {'ok': False, 'message': f'No existe el ID {d["id"]}'}

```